# Pandas

## Learning Objectives

- Create & manipulate Pandas DataFrames
- Load data file(s) into a Pandas DataFrame
- Index and subset Pandas DataFrames
- Generate descriptive statistics for data in Pandas DataFrames

---

`pandas` is a library with high-level data structures and manipulation tools:

**DataFrame Object**

A Pandas DataFrame is a two-dimensional size-mutable, potentially heterogeneous tabular data structure that holds ***relational data***.

Data is aligned in a tabular fashion with labeled rows and columns like a spreadsheet or SQL table, or a dict of Series objects. Essentially, a Pandas DataFrame consists of three components, the data, rows, and columns.

Key takeaways:

- Represents a tabular, spreadsheet-like data structure
- Ordered collection of columns
- Each column can be a different value type (numeric, string, boolean, etc.)
- Holds relational data

| | strings | bools | floats | arrays |
|---|---|---|---|---|
| **0** | A | True | 0.1 | [0.147809994902244, 0.6615215227442305, 0.8573... |
| **1** | B | True | 0.2 | [0.33213302863981464, 0.07507017149677342, 0.1... |
| **2** | C | False | 0.3 | [0.11271182114901401, 0.8577870983839114, 0.15... |
| **3** | D | False | 0.4 | [0.1187604694161426, 0.9972584470511201, 0.167... |
| **4** | E | True | 0.5 | [0.6562683396468842, 0.26465500651759855, 0.44... |

## Relational Data

Pandas dataframes/tables typically contain relational data. Relational data is data that captures associations or relationships between data points. This is often expressed as a table with columns indicating the quantities related. For example, "First Names" are associated with "Last Names".

This is a table with two columns, one column for First Name and one column for Last Name. For the pedantic, a "relation" is a table with no duplicate entries.

| First Name | Last Name |
|---|---|
| John | Smith |
| Jane | Doe |

This is a table with two columns, one column for First Name and one column for Last Name.

For the pedantic, a "relation" is a table with no duplicate entries. We might, for example, have two John Smiths. We can try to eliminate this collision problem by introducing an **Index**.

| Index | First Name | Last Name |
|---|---|---|
| **23452345** | John | Smith |
| **23434454** | Jane | Doe |

**A brief note on row vs index:** In this notebook and lecture we will use the terms row number and index. For many contexts these terms are interchangable, however due to pandas nomenclature around different ways of slicing and dicing data, we will have distinct meanings for these

**row number:** the number of the row starting with row 0

**index:** label given to a specific row

| Index | Column 1 | Column 2 |
|---|---|---|
| **Index 1** | A | B |
| **Index 2** | C | D |

In the above table we have added an "Index". The DataFrame object in Python is a representation of a table with these components. It is composed of rows, each with an index, and labeled columns.

In a general DataFrame, we might have many different relations captured in the same table. For example, a DataFrame with student data from a school might look something like this:

| Student ID | First Name | Last Name | Grade | School | Course | Gender |
|---|---|---|---|---|---|---|
| **23452345** | John | Smith | B | University of Washington | History | M |
| **23434454** | Jane | Doe | A | Stanford University | Physics | F |

### Data Representation

When thinking about data analysis, note that the above table already gives us something to think about regarding how choices of data representation affect conclusions.

- What if someone only has one name?
- What if they have a name that isn't easily represented as "First name/Last name"?
- What if they come from a culture that keeps track of multiple names?

Note there is a Gender column. What if the person who constructed this data had created this column as `isFemale`?

When interacting with tables and DataFrames, it is important to keep these issues in mind. Structural choices about data can and will affect conclusions. Whenever you make a DataFrame or use one someone else has constructed for you, you are making or dealing with these kinds of choices.

There are standard operations related to questions you might have about the students in the above DataFrame.

- Which students took Physics?
- Which students got an A in any course? Which Students from the University of Washington got an A or B in either Physics or History?

The DataFrame object has operations that allow these kinds of questions to be answered in a computationally efficient manner. These are covered in this tutorial.

## Many different relations

If we only needed to subselect from one table, we wouldn't really need something like `pandas` and its DataFrame (though it is helpful for this!). The real power of the DataFrame object appears when we have multiple relations, i.e. multiple DataFrames and we wish to combine them in some way.

For example we might have DataFrames that represent student Grades, or Professors, or Schools.

| Student ID | Grade |
|---|---|
| 74985674 | A |
| 26354957 | C |
| 62523934 | B |

or a DataFrame with Courses offered by Departments in different schools:

| | School | Department | Course | City | State |
|---|---|---|---|---|---|
| 456 | University of Washington | Physics | Quantum Mechanics | Seattle | WA |
| 894 | University of Chicago | Biology | Evolution and Ecology | Chicago | IL |

or a DataFrame of Professors and the Courses they teach

| Employee ID | Professor | Department | Course |
|---|---|---|---|
| 156354 | Emmy Nother | Physics | Quantum Mechanics |
| 987654 | Jane Goodall | Biology | Primatology |

The main purpose of `pandas` and the DataFrame object is to allow us to ask questions across multiple sets of relations.

- What is the average score of students of course Y from professor X (who may have taught at different institutions....)?
- What is the average number of students at school X from State Y?
- What is the distribution of grades from students in Biology whose home town is Y?

DataFrames have powerful tools like `merge` to combine information from multiple DataFrames that allow you to ask these kinds of questions quickly and easily.

Let's get started!

## Why Use Pandas

**Annotated Data is Powerful!**

Because pandas combines data labels with values it facilitites easy:

- Dataset exploration
- Dataset visualization
- Basic statistical analysis

**Data Manipulation is Easy!**

Pandas takes the functionality of slicing & dicing data and layers in many other helpful features that help with:

- Cleaning data
  - Column headers are descriptive not numerical
  - Columns hold single variables
  - Variables are stored in either rows or columns, not both
  - Handling missing or invalid values
- Data wrangling
  - Getting the data into a structure that facilitates your analysis
- Easily loading and saving data
  - Load many formats of data and save in many formats

**High Level Data Manipulation**

Pandas supports vectorized mathematical operations which optimizes computation performance and execution speed. Additionally the 2D labeled tabular structure of a pandas DataFrame allows other high level manipulations such as:

- Data grouping aggregation
- Table manipulations (transforming rows to columns and/or columns to rows)
- Merging or joinng multiple DataFrames/tables

---

## Documentation & Resources

> This introduction will only just scratch the surface of Pandas functionality. For more information, check out the full documentation
>
> This cheat-sheet is also highly recommended to print out and keep handy as a resource.
>
> Or check out the '10 minutes to Pandas' tutorial here (note: title may mischaracterize time investment).

---

# Import Packages

**Library imports**
Here we'll load in the libraries we'll use to shape and explore the data

In [1]:
```
import os
import numpy as np
```

In [2]:
```
# import `pandas` and give it a short name `pd` since we will type it freque
import pandas as pd
```

## Load dataset

Our first step is loading in the data from a file

Pandas has great tools for automatically interpretting data from many sources

- pd.read_csv
- pd.read_feather
- pd.read_excel
- pd.read_pickle
- pd.read_json

We will use `pd.read_feather`

In [3]:
```
# Create directory structure if it doesn't exist
os.makedirs('support_files/datasets', exist_ok=True)

# Download from GitHub
!wget -O support_files/datasets/messy_superstore_data.feather https://github

# Load file
filepath = os.path.join('support_files', 'datasets', 'messy_superstore_data.
df = pd.read_feather(filepath)
```

```
# Set the index
df.set_index('Row ID', inplace=True)
```

```
--2026-07-24 16:20:25--  https://github.com/AllenInstitute/EducationAndEngag
ement/raw/main/NeuroCodingWorkshop/support_files/datasets/messy_superstore_d
ata.feather
Resolving github.com (github.com)... 140.82.113.3
Connecting to github.com (github.com)|140.82.113.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/AllenInstitute/EducationAndEngag
ement/main/NeuroCodingWorkshop/support_files/datasets/messy_superstore_data.
feather [following]
--2026-07-24 16:20:25--  https://raw.githubusercontent.com/AllenInstitute/Ed
ucationAndEngagement/main/NeuroCodingWorkshop/support_files/datasets/messy_s
uperstore_data.feather
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.1
08.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.
108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2891970 (2.8M) [application/octet-stream]
Saving to: ‘support_files/datasets/messy_superstore_data.feather’

support_files/datas 100%[===================>]   2.76M  --.-KB/s    in 0.09s

2026-07-24 16:20:26 (31.4 MB/s) - ‘support_files/datasets/messy_superstore_d
ata.feather’ saved [2891970/2891970]
```

---

# Explore the dataset

## Basic data exploration

- view dataframe "preview"
- view rows from beginning or end of dataframe ( `df.head` , `df.tail` )
- get dataframe shape and length ( `len` , `np.shape` )
- list all columns ( `.columns` )
- show single column data
- get column data types ( `.dtypes` )
- get unique entries ( `.unique()` )

## Basic data visualization

- plotting with pandas ( `.plot` )

## Descriptive statistics

- get table level descriptive statistics

- other built in summary functions

---

### View the DataFrame

Simply calling the DataFrame ( `df` or whatever you've named the DataFrame) gives a preview view of the DataFrame/table

- shows the first 5 and last 5 rows of data
- shows the first 10 and last 10 columns of data

In [4]:
```
# view the df

df
```

Out[4]:

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | Produ |
|---|---|---|---|---|---|---|---|
| **IN-2014-23218** | IN-2014-75456 | Consumer | FURNITURE | NaN | FURNISHINGS | Rubbermaid Door Stop, Erganomic | FUR-F 100040 |
| **IN-2014-24599** | IN-2014-29767 | Home Office | FURNITURE | NaN | BOOKCASES | Ikea Library with Doors, Mobile | FUR-B 100012 |
| **IN-2014-24597** | IN-2014-29767 | Home Office | FURNITURE | NaN | FURNISHINGS | Rubbermaid Door Stop, Erganomic | FUR-F 100040 |
| **IN-2014-27993** | IN-2014-20415 | Home Office | FURNITURE | NaN | BOOKCASES | Bush Classic Bookcase, Pine | FUR-B 100022 |
| **IN-2014-28967** | IN-2014-47337 | Corporate | FURNITURE | NaN | CHAIRS | Hon Rocking Chair, Red | FUR-C 100039 |
| **...** | ... | ... | ... | ... | ... | ... | |
| **ZA-2014-49187** | ZA-2014-9750 | Corporate | TECHNOLOGY | NaN | ACCESSORIES | Memorex Router, USB | TE ME 100022 |
| **ZI-2014-42069** | ZI-2014-7610 | Corporate | TECHNOLOGY | NaN | MACHINES | StarTech Phone, Red | TEC-ST 100006 |
| **ZI-2014-43712** | ZI-2014-5970 | Home Office | TECHNOLOGY | NaN | ACCESSORIES | Belkin Router, USB | TEC-BI 100039 |
| **ZI-2014-48372** | ZI-2014-9550 | Consumer | TECHNOLOGY | NaN | MACHINES | Konica Receipt Printer, Red | TE KC 10003 |
| **ZI-2014-48014** | ZI-2014-3570 | Consumer | TECHNOLOGY | NaN | MACHINES | Okidata Calculator, Red | TEC-O 100014 |

17531 rows × 34 columns

## View the first or last n rows

**.head()**: shows the first n rows

**.tail()**: shows the last n rows

- `df.head()` and `df.tail()` shows 5 rows of data by default
- adding a number `df.head(n)` adjusts the number of rows shown

In [5]:
```
# add a number to show the first `n` rows

df.head()
```

Out[5]:

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | Product ID |
|---|---|---|---|---|---|---|---|
| **IN-2014-23218** | IN-2014-75456 | Consumer | FURNITURE | NaN | FURNISHINGS | Rubbermaid Door Stop, Erganomic | FUR-FU-10004064 |
| **IN-2014-24599** | IN-2014-29767 | Home Office | FURNITURE | NaN | BOOKCASES | Ikea Library with Doors, Mobile | FUR-BO-10001255 |
| **IN-2014-24597** | IN-2014-29767 | Home Office | FURNITURE | NaN | FURNISHINGS | Rubbermaid Door Stop, Erganomic | FUR-FU-10004064 |
| **IN-2014-27993** | IN-2014-20415 | Home Office | FURNITURE | NaN | BOOKCASES | Bush Classic Bookcase, Pine | FUR-BO-10002204 |
| **IN-2014-28967** | IN-2014-47337 | Corporate | FURNITURE | NaN | CHAIRS | Hon Rocking Chair, Red | FUR-CH-10003965 |

5 rows × 34 columns

In [6]:
```
# add a number to show the last `n` rows

df.tail()
```

Out[6]:

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | Produc Il |
|---|---|---|---|---|---|---|---|
| **ZA-2014-49187** | ZA-2014-9750 | Corporate | TECHNOLOGY | NaN | ACCESSORIES | Memorex Router, USB | TEC MEM 1000220 |
| **ZI-2014-42069** | ZI-2014-7610 | Corporate | TECHNOLOGY | NaN | MACHINES | StarTech Phone, Red | TEC-STA 1000069 |
| **ZI-2014-43712** | ZI-2014-5970 | Home Office | TECHNOLOGY | NaN | ACCESSORIES | Belkin Router, USB | TEC-BEL 1000398 |
| **ZI-2014-48372** | ZI-2014-9550 | Consumer | TECHNOLOGY | NaN | MACHINES | Konica Receipt Printer, Red | TEC KON 1000311 |
| **ZI-2014-48014** | ZI-2014-3570 | Consumer | TECHNOLOGY | NaN | MACHINES | Okidata Calculator, Red | TEC-OKI 1000143 |

5 rows × 34 columns

### Get the shape and length of the DataFrame

pandas is built off of numpy, so many familiar functions/methods work with DataFrames

- **numpys shape function** can give the dimensions of a DataFrame
  - `df.shape` - returns (rows, columns)
- **len()** - the built in python function will return the number of rows in a DataFrame
  - `len(df)`

In [7]:
```
# code goes here to find the shape of the df

df.shape
```

Out[7]:
```
(17531, 34)
```

In [8]:
```
# code goes here to find the length of the df
```

```
len(df)
```

Out[8]: 17531

### List Columns

**.columns** provides a list of the column labels for a DataFrame

`df.columns`

In [9]:
```
# let's try it out

df.columns
```

Out[9]:
```
Index(['Order ID', 'Segment', 'Category', 'Category (OLD)', 'Sub-Category',
       'Product Name', 'Product ID', 'Country', 'Market', 'Region', 'Quanti
ty',
       'Discount', 'Profit', 'Customer ID', 'Customer Name', 'Order Priorit
y',
       'Postal Code', 'Ship Mode', 'Shipping Cost', '10/1/2014', '7/1/201
4',
       '11/1/2014', '9/1/2014', '1/1/2014', '12/1/2014', '8/1/2014',
       '5/1/2014', '3/1/2014', '4/1/2014', '2/1/2014', '6/1/2014', 'City',
       'State', 'manufacturers'],
      dtype='object')
```

### Show data from a single column

Like retrieving a value from a dictionary, we can get the data for a single column by indexing with the column's name:

In [10]:
```
df["Country"]
```

Out[10]:

| Row ID | Country |
|---|---|
| **IN-2014-23218** | Afghanistan |
| **IN-2014-24599** | Afghanistan |
| **IN-2014-24597** | Afghanistan |
| **IN-2014-27993** | Afghanistan |
| **IN-2014-28967** | Afghanistan |
| **...** | ... |
| **ZA-2014-49187** | Zambia |
| **ZI-2014-42069** | Zimbabwe |
| **ZI-2014-43712** | Zimbabwe |
| **ZI-2014-48372** | Zimbabwe |
| **ZI-2014-48014** | Zimbabwe |

17531 rows × 1 columns

**dtype:** object

### Get column or element data types

- **.dtypes** : lists the data type for all columns in a DataFrame
  - `df.dtypes`
- **type()** allows inspection of data type for a specific element (i.e. specific row & column)
  - this can be helpful if the `.dtypes` function returns "object"

In [11…

```
df.dtypes
```

Out[11]:

| | 0 |
|---|---|
| **Order ID** | object |
| **Segment** | object |
| **Category** | object |
| **Category (OLD)** | float64 |
| **Sub-Category** | object |
| **Product Name** | object |
| **Product ID** | object |
| **Country** | object |
| **Market** | object |
| **Region** | object |
| **Quantity** | float64 |
| **Discount** | float64 |
| **Profit** | float64 |
| **Customer ID** | object |
| **Customer Name** | object |
| **Order Priority** | object |
| **Postal Code** | float64 |
| **Ship Mode** | object |
| **Shipping Cost** | float64 |
| **10/1/2014** | float64 |
| **7/1/2014** | float64 |
| **11/1/2014** | float64 |
| **9/1/2014** | float64 |
| **1/1/2014** | float64 |
| **12/1/2014** | float64 |
| **8/1/2014** | float64 |
| **5/1/2014** | float64 |
| **3/1/2014** | float64 |
| **4/1/2014** | float64 |
| **2/1/2014** | float64 |
| **6/1/2014** | float64 |
| **City** | object |

| | 0 |
|---|---|
| **State** | object |
| **manufacturers** | object |

**dtype:** object

In [12…
```
# use the built in type() function to get the first row of the "Category" c

# first find the row info
df["Category"]
```

Out[12]:

| Row ID | Category |
|---|---|
| **IN-2014-23218** | FURNITURE |
| **IN-2014-24599** | FURNITURE |
| **IN-2014-24597** | FURNITURE |
| **IN-2014-27993** | FURNITURE |
| **IN-2014-28967** | FURNITURE |
| **…** | … |
| **ZA-2014-49187** | TECHNOLOGY |
| **ZI-2014-42069** | TECHNOLOGY |
| **ZI-2014-43712** | TECHNOLOGY |
| **ZI-2014-48372** | TECHNOLOGY |
| **ZI-2014-48014** | TECHNOLOGY |

17531 rows × 1 columns

**dtype:** object

In [13…
```
# use type()

type(df["Category"]["IN-2014-23218"])
```

Out[13]:
```
str
```

Get unique entries for a column

`.unique`

`df['column_name'].unique()` returns an array of all unique entries

In [14…
```
# get all unique entries for the Country column

df["Country"].unique()
```

Out[14]:
```
array(['Afghanistan', 'Algeria', 'Angola', 'Argentina', 'Australia',
       'Austria', 'Azerbaijan', 'Bangladesh', 'Barbados', 'Belarus',
       'Belgium', 'Bolivia', 'Brazil', 'Bulgaria', 'Cambodia', 'Cameroon',
       'Canada', 'Chile', 'China', 'Colombia', "Cote d'Ivoire", 'Croatia',
       'Cuba', 'Czech Republic', 'Democratic Republic of the Congo',
       'Denmark', 'Dominican Republic', 'Ecuador', 'Egypt', 'El Salvador',
       'Estonia', 'Finland', 'France', 'Gabon', 'Georgia', 'Germany',
       'Ghana', 'Guatemala', 'Haiti', 'Honduras', 'Hungary', 'India',
       'Indonesia', 'Iran', 'Iraq', 'Ireland', 'Israel', 'Italy',
       'Jamaica', 'Japan', 'Jordan', 'Kazakhstan', 'Kenya', 'Kyrgyzstan',
       'Lebanon', 'Liberia', 'Libya', 'Lithuania', 'Macedonia',
       'Madagascar', 'Malaysia', 'Mali', 'Martinique', 'Mexico',
       'Moldova', 'Mongolia', 'Montenegro', 'Morocco', 'Mozambique',
       'Myanmar (Burma)', 'Nepal', 'Netherlands', 'New Zealand',
       'Nicaragua', 'Niger', 'Nigeria', 'Norway', 'Pakistan', 'Panama',
       'Paraguay', 'Peru', 'Philippines', 'Poland', 'Portugal',
       'Republic of the Congo', 'Romania', 'Russia', 'Rwanda',
       'Saudi Arabia', 'Senegal', 'Singapore', 'Slovenia', 'Somalia',
       'South Africa', 'South Korea', 'Spain', 'Sudan', 'Sweden',
       'Switzerland', 'Syria', 'Tanzania', 'Thailand', 'Togo', 'Tunisia',
       'Turkey', 'Turkmenistan', 'Uganda', 'Ukraine', 'United Kingdom',
       'United States', 'Uruguay', 'Uzbekistan', 'Venezuela', 'Vietnam',
       'Yemen', 'Zambia', 'Zimbabwe', 'Albania', 'Benin',
       'Bosnia and Herzegovina', 'Central African Republic', 'Ethiopia',
       'Guinea', 'Guinea-Bissau', 'Hong Kong', 'Mauritania', 'Namibia',
       'Papua New Guinea', 'Qatar', 'Sierra Leone', 'Slovakia',
       'Sri Lanka', 'Taiwan', 'Tajikistan', 'Trinidad and Tobago',
       'United Arab Emirates', 'South Sudan', 'Swaziland'], dtype=object)
```

**Exercise:**

1. What are the `unique` values for the Market column?
2. Identify the data `type` for the Product ID column

In [15…
```
### Your code here
```

In [16…
```
### Your code here
```

Solutions

In [17…
```
df["Market"].unique()
```

Out[17]:
```
array(['APAC', 'Africa', 'LATAM', 'EU', 'EMEA', 'Canada', 'US'],
      dtype=object)
```

In [18…
```
df.Market.unique()
```

Out[18]:
```
array(['APAC', 'Africa', 'LATAM', 'EU', 'EMEA', 'Canada', 'US'],
      dtype=object)
```

In [19…
```
df["Product ID"]
```

Out[19]:

| Row ID | Product ID |
|---|---|
| **IN-2014-23218** | FUR-FU-10004064 |
| **IN-2014-24599** | FUR-BO-10001255 |
| **IN-2014-24597** | FUR-FU-10004064 |
| **IN-2014-27993** | FUR-BO-10002204 |
| **IN-2014-28967** | FUR-CH-10003965 |
| **…** | … |
| **ZA-2014-49187** | TEC-MEM-10002202 |
| **ZI-2014-42069** | TEC-STA-10000699 |
| **ZI-2014-43712** | TEC-BEL-10003985 |
| **ZI-2014-48372** | TEC-KON-10003116 |
| **ZI-2014-48014** | TEC-OKI-10001433 |

17531 rows × 1 columns

**dtype:** object

Note the singular versus plural form here

In [20…
```
type(df["Product ID"]["IN-2014-23218"])
```

Out[20]:
```
str
```

In [21…
```
type(df["Product ID"][0])
```

```
/tmp/ipykernel_1818/3651682204.py:1: FutureWarning: Series.__getitem__ trea
ting keys as positions is deprecated. In a future version, integer keys wil
l always be treated as labels (consistent with DataFrame behavior). To acce
ss a value by position, use `ser.iloc[pos]`
  type(df["Product ID"][0])
```

Out[21]:
```
str
```

In [22…
```
df["Product ID"].dtype
```

Out[22]:
```
dtype('O')
```

In [23…
```
print(df["Product ID"].dtype)
```

```
object
```

### Plotting with Pandas!

Pandas has a set of plotting tools built into the library. You can quickly create visualizations (especially with grouped and aggregated data) using pandas built in plotting functions such as:

- `.plot`

Example:

```
df.plot(x = 'xcolumn',
        y = 'ycolumn',
        kind = "box",
        xlabel= 'x value units',
        ylabel = 'y value units')
```

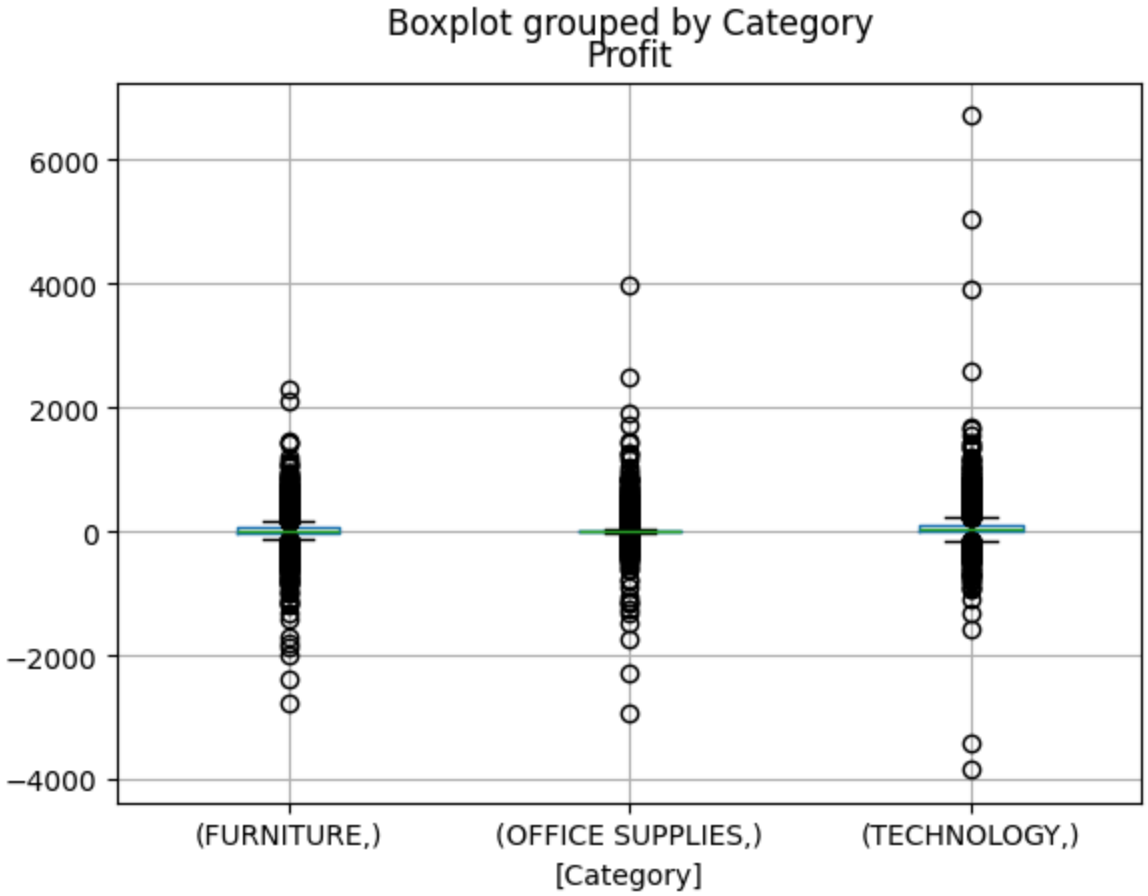
- `.boxplot`

**NOTE:** Pandas works especially well with the the `seaborn` plotting package. We will cover this later during other sessions!

In [24…
```
# let's create a boxplot of profit by category

df.boxplot(column = ["Profit"], by = ["Category"])
```

Out[24]:
```
<Axes: title={'center': 'Profit'}, xlabel='[Category]'>
```




## Get descriptive statistics

Descriptive statistic are summary statistics that quantitatively describes or summarizes features from a collection of information or dataset. This typically included things like sample size, measures of central tendency (mean, median, mode), measures of variability or dispersion (standard deviation, min, max, kurtosis, skewness)

`.describe()` will return descriptive statistics for all quantitative columns of a DataFrame

`df.describe()`

For each numerical column the following descriptive statistics are provided:

- count

- mean
- standard deviation
- minimum
- 25, 50 & 75th percentiles
- max

In [25…
```
# try out describe on our df

df.describe()
```

Out[25]:

| | Category (OLD) | Quantity | Discount | Profit | Postal Code | Shi |
|---|---|---|---|---|---|---|
| **count** | 0.0 | 17531.000000 | 17531.000000 | 17531.000000 | 3321.000000 | 17531.0( |
| **mean** | NaN | 3.457989 | 0.143291 | 28.758540 | 56192.587775 | 26.26 |
| **std** | NaN | 2.290856 | 0.211830 | 174.283412 | 31977.397359 | 56.54 |
| **min** | NaN | 1.000000 | 0.000000 | -3839.990400 | 1841.000000 | 0.0 |
| **25%** | NaN | 2.000000 | 0.000000 | 0.000000 | 28110.000000 | 2.57 |
| **50%** | NaN | 3.000000 | 0.000000 | 9.200000 | 60440.000000 | 7.72 |
| **75%** | NaN | 5.000000 | 0.200000 | 36.808500 | 90032.000000 | 24.45 |
| **max** | NaN | 14.000000 | 0.800000 | 6719.980800 | 99301.000000 | 867.69 |

### Other built in summary statistic functions

Pandas also provides a large set of summary functions that can operate on different kinds of pandas objects (DataFrame columns, series, groupby etc.)

- `.count()`
- `.sum()`
- `.min()`
- `.max()`
- `.mean()`
- `.median()`
- `.var()`
- `.std()`

These functions become especially useful in the next section where there might be some specific selection of data you want to analyze

In [26…
```
# here we get the mean discount for the entire column

df["Discount"].mean()
```

Out[26]:
```
np.float64(0.14329119844846275)
```

**Exercise:**

1. What is the **maximum** value in the Shipping Cost column?

In [27…
```
### Your code here
```

Solution

In [28…
```
df["Shipping Cost"].max()
```

Out[28]:
```
867.69
```

---

# Data Selection, Slicing & Dicing

## Basic data selection & filtering

- column selection
- select data by labels/names
- select data by integer index/position

## Advanced data selection

- select data for a given condition or threshold
- return subset of data for a given condition or threshold
- select data based on multiple conditions or thresholds

## Note

- 'view' of data vs a copy

### Column Selection

**column selection**

To view a single single column:

- `df['column_name']`
  - returns a pandas Series object, which is similar to a 1D array or list

~ OR ~

- `df[['column_name']]`
  - returns a DataFrame

Multiple columns:

- `df[['column_1', 'column_2']]`

In [29…
```
# get the Category column as a pd.series

df["Category"]
```

Out[29]:

| Row ID | Category |
|---|---|
| **IN-2014-23218** | FURNITURE |
| **IN-2014-24599** | FURNITURE |
| **IN-2014-24597** | FURNITURE |
| **IN-2014-27993** | FURNITURE |
| **IN-2014-28967** | FURNITURE |
| **…** | … |
| **ZA-2014-49187** | TECHNOLOGY |
| **ZI-2014-42069** | TECHNOLOGY |
| **ZI-2014-43712** | TECHNOLOGY |
| **ZI-2014-48372** | TECHNOLOGY |
| **ZI-2014-48014** | TECHNOLOGY |

17531 rows × 1 columns

**dtype:** object

In [30…
```
type(df["Category"])
```

Out[30]:

**pandas.core.series.Series**

def __init__(data=None, index=None, dtype: Dtype | None=None, name=None, copy: bool | None=None, fastpath: bool | lib.NoDefault=lib.no_default) -> None

One-dimensional ndarray with axis labels (including time series).

Labels need not be unique but must be a hashable type. The object supports both integer- and label-based indexing and provides a host of methods for performing operations involving the index. Statistical

In [31…

```
# get the Category and Customer ID columns as a dataframe

df[["Category", "Customer ID"]]
```

Out[31]:

| Row ID | Category | Customer ID |
|---|---|---|
| **IN-2014-23218** | FURNITURE | AA-10375 |
| **IN-2014-24599** | FURNITURE | CA-12055 |
| **IN-2014-24597** | FURNITURE | CA-12055 |
| **IN-2014-27993** | FURNITURE | GM-14455 |
| **IN-2014-28967** | FURNITURE | VB-21745 |
| **...** | ... | ... |
| **ZA-2014-49187** | TECHNOLOGY | TS-11205 |
| **ZI-2014-42069** | TECHNOLOGY | BS-1380 |
| **ZI-2014-43712** | TECHNOLOGY | JB-6045 |
| **ZI-2014-48372** | TECHNOLOGY | JC-5775 |
| **ZI-2014-48014** | TECHNOLOGY | NG-8430 |

17531 rows × 2 columns

In [32…

```
type(df[["Category", "Customer ID"]])
```

Out[32]:

**pandas.core.frame.DataFrame**

def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype | None=None, copy: bool | None=None) -> None

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns).
Arithmetic operations align on both row and column labels. Can be
thought of as a dict-like container for Series objects. The primary

### Select rows or columns using labels

`.loc()` allows access to a group of rows and columns by label(s) or a boolean array.

**select rows:**

- `df.loc['index']`
  - Hint: similar to column selection described above, for row selection using `df.loc[ ]` returns a Series where `df.loc[[ ]]` returns a DataFrame

**select row and column:**

- `df.loc['index', 'column']`

In [33…

```
# get rows where the index is IN-2014-28967

df.loc["IN-2014-28967"]
```

Out[33]:

| | IN-2014-28967 |
|---|---|
| **Order ID** | IN-2014-47337 |
| **Segment** | Corporate |
| **Category** | FURNITURE |
| **Category (OLD)** | NaN |
| **Sub-Category** | CHAIRS |
| **Product Name** | Hon Rocking Chair, Red |
| **Product ID** | FUR-CH-10003965 |
| **Country** | Afghanistan |
| **Market** | APAC |
| **Region** | Central Asia |
| **Quantity** | 7.0 |
| **Discount** | 0.0 |
| **Profit** | 356.58 |
| **Customer ID** | VB-21745 |
| **Customer Name** | Victoria Brennan |
| **Order Priority** | High |
| **Postal Code** | NaN |
| **Ship Mode** | Standard Class |
| **Shipping Cost** | 106.41 |
| **10/1/2014** | NaN |
| **7/1/2014** | NaN |
| **11/1/2014** | NaN |
| **9/1/2014** | NaN |
| **1/1/2014** | NaN |
| **12/1/2014** | 914.34 |
| **8/1/2014** | NaN |
| **5/1/2014** | NaN |
| **3/1/2014** | NaN |
| **4/1/2014** | NaN |
| **2/1/2014** | NaN |
| **6/1/2014** | NaN |
| **City** | Kabul |

| | IN-2014-28967 |
|---|---|
| **State** | Kabul |
| **manufacturers** | [ACME Co, Buy n Large, Dunder Mifflin, LexCorp... |

**dtype:** object

In [34…
```
df.loc[["IN-2014-28967"]]
```

Out[34]:

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | Product ID | |
|---|---|---|---|---|---|---|---|---|
| **IN-2014-28967** | IN-2014-47337 | Corporate | FURNITURE | NaN | CHAIRS | Hon Rocking Chair, Red | FUR-CH-10003965 | Afg |

1 rows × 34 columns

In [35…
```
# get index:ZI-2014-48372 and column "Country"

df.loc["ZI-2014-48372", "Country"]
```

Out[35]:
```
'Zimbabwe'
```

## Select rows and columns by position

**.iloc()** allows positional selection: row number, column number

(Note: the same slicing notation that we saw with `numpy` arrays works with `iloc`.)

Example: `df.iloc([row(s) number, column(s) number])`

Examples:

- Select all rows and all columns
  - `df.iloc[:,:]`
- Select first 5 rows and all columns
  - `df.iloc[0:4, :]`
- Select all rows and last 5 columns
  - `.iloc[:,-5:]`

In [36…

```
# get the first 9 rows and all columns

df.iloc[:9, :]
```

Out[36]:

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | Prod |
|---|---|---|---|---|---|---|---|
| **IN-2014-23218** | IN-2014-75456 | Consumer | FURNITURE | NaN | FURNISHINGS | Rubbermaid Door Stop, Erganomic | FUR-I 10004( |
| **IN-2014-24599** | IN-2014-29767 | Home Office | FURNITURE | NaN | BOOKCASES | Ikea Library with Doors, Mobile | FUR-E 10001 |
| **IN-2014-24597** | IN-2014-29767 | Home Office | FURNITURE | NaN | FURNISHINGS | Rubbermaid Door Stop, Erganomic | FUR-I 10004( |
| **IN-2014-27993** | IN-2014-20415 | Home Office | FURNITURE | NaN | BOOKCASES | Bush Classic Bookcase, Pine | FUR-E 10002 |
| **IN-2014-28967** | IN-2014-47337 | Corporate | FURNITURE | NaN | CHAIRS | Hon Rocking Chair, Red | FUR-( 10003 |
| **AG-2014-50986** | AG-2014-2760 | Consumer | FURNITURE | NaN | FURNISHINGS | Deflect-O Light Bulb, Erganomic | FUR-D 10002 |
| **AG-2014-50983** | AG-2014-2760 | Consumer | FURNITURE | NaN | CHAIRS | Novimex Rocking Chair, Black | FU N( 10002 |
| **AG-2014-49384** | AG-2014-2040 | Consumer | FURNITURE | NaN | FURNISHINGS | Rubbermaid Frame, Durable | FU RU 10003( |
| **AG-2014-47438** | AG-2014-2600 | Home Office | FURNITURE | NaN | BOOKCASES | Ikea 3-Shelf Cabinet, Pine | FUR-I 10003 |

9 rows × 34 columns

In [37…
```
# get all rows and last 4 columns

df.iloc[:, -4:]
```

Out[37]:

| Row ID | 6/1/2014 | City | State | manufacturers |
|---|---|---|---|---|
| **IN-2014-23218** | NaN | Kabul | Kabul | [Dunder Mifflin, Globex Corp, Hudsucker Indust... |
| **IN-2014-24599** | NaN | Herat | Hirat | [ACME Co, Buy n Large, Dunder Mifflin, Globex ... |
| **IN-2014-24597** | NaN | Herat | Hirat | [Dunder Mifflin, LexCorp, Olivander Crafts, Ro... |
| **IN-2014-27993** | NaN | Kabul | Kabul | [Dunder Mifflin, Olivander Crafts] |
| **IN-2014-28967** | NaN | Kabul | Kabul | [ACME Co, Buy n Large, Dunder Mifflin, LexCorp... |
| **...** | ... | ... | ... | ... |
| **ZA-2014-49187** | NaN | Ndola | Copperbelt | [ACME Co, Royco Waystar, Umbrella Corporation] |
| **ZI-2014-42069** | NaN | Bulawayo | Bulawayo | [Dunder Mifflin] |
| **ZI-2014-43712** | 77.688 | Bulawayo | Bulawayo | [ACME Co, Buy n Large, Hudsucker Industries] |
| **ZI-2014-48372** | NaN | Bulawayo | Bulawayo | [ACME Co, Wayne Enterprises] |
| **ZI-2014-48014** | NaN | Harare | Harare | [Dunder Mifflin, Olivander Crafts, Royco Wayst... |

17531 rows × 4 columns

## Advanced data selection

### Select data given a specific threshold or condition

You can conditionally select data using **.loc[ ]** or **.query()** or directly on the DataFrame. Each of these utilize boolean masking. `.loc`

- `df.loc[df['column'] > 7]`
- `df.loc[df['column'] == 'string']`
  - Pros: explicit, can be easier to read for those who already know python
  - Cons: more verbose than query

---

`.query()` : This method uses boolean expressions and may be easier & more intuitive for those who know SQL or other database languages.

Examples:

- `df.query('column > 7')`
- `df.query('column == string')`
  - Pros: can be easier for those who know SQL or other database languages, less verbose
  - Cons: can be difficult for those that don't know SQL, doesn't handle column names that contain spaces very well

In [38…

```
# use .loc to select where Profit > 500

df.loc[df["Profit"] > 500]
```

Out[38]:

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | I |
|---|---|---|---|---|---|---|---|
| **IN-2014-27993** | IN-2014-20415 | Home Office | FURNITURE | NaN | BOOKCASES | Bush Classic Bookcase, Pine | F 10 |
| **AO-2014-44396** | AO-2014-2270 | Consumer | FURNITURE | NaN | TABLES | Chromcraft Round Table, Adjustable Height | 10 |
| **IN-2014-24624** | IN-2014-23880 | Consumer | FURNITURE | NaN | BOOKCASES | Sauder Classic Bookcase, Traditional | F 10 |
| **IN-2014-31153** | IN-2014-80286 | Consumer | FURNITURE | NaN | CHAIRS | Harbour Creations Executive Leather Armchair, ... | F 10 |
| **ID-2014-21496** | ID-2014-35892 | Consumer | FURNITURE | NaN | CHAIRS | SAFCO Executive Leather Armchair, Set of Two | F 10 |
| **...** | ... | ... | ... | ... | ... | ... | |
| **CA-2014-33921** | CA-2014-127180 | Home Office | TECHNOLOGY | NaN | PHONES | Polycom CX600 IP Phone VoIP phone | T 10 |
| **CA-2014-33920** | CA-2014-127180 | Home Office | TECHNOLOGY | NaN | COPIERS | Canon imageCLASS 2200 Advanced Copier | T 10 |
| **CA-2014-36178** | CA-2014-143567 | Corporate | TECHNOLOGY | NaN | ACCESSORIES | Logitech diNovo Edge Keyboard | T 10 |
| **CA-2014-37637** | CA-2014-143112 | Corporate | TECHNOLOGY | NaN | MACHINES | 3D Systems Cube Printer, 2nd Generation, Magenta | T 10 |
| **CA-2014-38429** | CA-2014-141439 | Home Office | TECHNOLOGY | NaN | PHONES | Samsung Galaxy S4 Mini | T 10 |

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | |
|---|---|---|---|---|---|---|---|

231 rows × 34 columns

### Return a subset of data given a specific threshold or condition

You can also return specific columns after doing conditional selection data using `.loc` .

Single return column:

- `df.loc[df['selection_column'] > 7, 'return_column']`

Multiple return columns (provide columns as a list):

- `df.loc[df['selection_column'] > 7, ['return_column1', 'return_column2']]`

In [39…

```
# get the Category and Sub-Category where profit is above 500

df.loc[df["Profit"] > 500, ["Category", "Sub-Category"]]
```

Out[39]:

| Row ID | Category | Sub-Category |
|---|---|---|
| **IN-2014-27993** | FURNITURE | BOOKCASES |
| **AO-2014-44396** | FURNITURE | TABLES |
| **IN-2014-24624** | FURNITURE | BOOKCASES |
| **IN-2014-31153** | FURNITURE | CHAIRS |
| **ID-2014-21496** | FURNITURE | CHAIRS |
| **...** | ... | ... |
| **CA-2014-33921** | TECHNOLOGY | PHONES |
| **CA-2014-33920** | TECHNOLOGY | COPIERS |
| **CA-2014-36178** | TECHNOLOGY | ACCESSORIES |
| **CA-2014-37637** | TECHNOLOGY | MACHINES |
| **CA-2014-38429** | TECHNOLOGY | PHONES |

231 rows × 2 columns

### Multiple condition selection

You may wish to select rows or columns where multiple conditions are met. You can combine conditions with `.loc` in `numpy` and utilize `&` and `|`

Example: `df.loc[(df['column1'] == 'string') & (df['column2'] > threshold)]`

In [40…

```
# return all entries where Profit is greater than 900 and the Sub-Categor
# the & means both conditions must be met

df.loc[(df["Sub-Category"] == "BOOKCASES") & (df["Profit"] > 900)]
```

Out[40]:

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | Produ |
|---|---|---|---|---|---|---|---|
| **IN-2014-24624** | IN-2014-23880 | Consumer | FURNITURE | NaN | BOOKCASES | Sauder Classic Bookcase, Traditional | FUR-B 100048 |
| **IN-2014-21263** | IN-2014-56206 | Consumer | FURNITURE | NaN | BOOKCASES | Sauder Classic Bookcase, Metal | FUR-B 100014 |
| **IN-2014-24037** | IN-2014-17930 | Consumer | FURNITURE | NaN | BOOKCASES | Sauder Classic Bookcase, Metal | FUR-B 100014 |
| **IN-2014-28056** | IN-2014-66573 | Consumer | FURNITURE | NaN | BOOKCASES | Bush Classic Bookcase, Mobile | FUR-B 100046 |
| **MX-2014-4452** | MX-2014-157077 | Consumer | FURNITURE | NaN | BOOKCASES | Dania Classic Bookcase, Traditional | FUR-B 100023 |
| **IN-2014-25795** | IN-2014-76016 | Corporate | FURNITURE | NaN | BOOKCASES | Sauder Classic Bookcase, Traditional | FUR-B 100048 |
| **IT-2014-16653** | IT-2014-4540740 | Consumer | FURNITURE | NaN | BOOKCASES | Safco Classic Bookcase, Metal | FUR-B 100049 |
| **ES-2014-12449** | ES-2014-4957212 | Consumer | FURNITURE | NaN | BOOKCASES | Bush Classic Bookcase, Pine | FUR-B 100047 |
| **ZA-2014-42448** | ZA-2014-7540 | Consumer | FURNITURE | NaN | BOOKCASES | Ikea Library with Doors, Traditional | FUR-IK 100028 |

9 rows × 34 columns

In [41…

```
# return all entries where the Country is "Australia" or the Country is "
# the | means at least one condition must be met
```

```
df.loc[(df["Country"] == "Australia") | (df["Country"] == "Zambia")]
```

Out[41]:

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | Prod |
|---|---|---|---|---|---|---|---|
| **ID-2014-25801** | ID-2014-42388 | Consumer | FURNITURE | NaN | FURNISHINGS | Deflect-O Frame, Duo Pack | FUR-10004 |
| **IN-2014-21802** | IN-2014-25644 | Consumer | FURNITURE | NaN | BOOKCASES | Bush Corner Shelving, Metal | FUR-10004 |
| **IN-2014-22521** | IN-2014-10090 | Consumer | FURNITURE | NaN | CHAIRS | Harbour Creations Steel Folding Chair, Red | FUR-10001 |
| **IN-2014-26860** | IN-2014-64396 | Home Office | FURNITURE | NaN | FURNISHINGS | Deflect-O Stacking Tray, Black | FUR-1000 |
| **ID-2014-30742** | ID-2014-84164 | Home Office | FURNITURE | NaN | FURNISHINGS | Tenex Photo Frame, Duo Pack | FUR-10000 |
| **...** | ... | ... | ... | ... | ... | ... | |
| **ZA-2014-41769** | ZA-2014-1370 | Consumer | TECHNOLOGY | NaN | COPIERS | Brother Personal Copier, Color | T B 10003 |
| **ZA-2014-45397** | ZA-2014-8330 | Corporate | TECHNOLOGY | NaN | PHONES | Cisco Headset, VoIP | TEC-( 10003 |
| **ZA-2014-50068** | ZA-2014-6660 | Home Office | TECHNOLOGY | NaN | COPIERS | Brother Fax and Copier, High-Speed | T B 10003 |
| **ZA-2014-50069** | ZA-2014-6660 | Home Office | TECHNOLOGY | NaN | COPIERS | Hewlett Fax Machine, High-Speed | T H 10002 |
| **ZA-2014-** | ZA-2014- | Corporate | TECHNOLOGY | NaN | ACCESSORIES | Memorex Router, | T M |

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | Proc |
|---|---|---|---|---|---|---|---|
| **49187** | 9750 | | | | | USB | 10002 |

1023 rows × 34 columns

**Exercises:**

1. Select Row ID "AG-2014-50983" and all columns
2. Get the mean profit when the Category is furniture
3. How many entries in the DataFrame are there where Profit is negative and Market is EU

In [42…
```
### Your code here
```

In [43…
```
### Your code here
```

In [44…
```
### Your code here
```

Solutions

In [45…
```
# row id AG-2014-50983 and all cols

df.loc[["AG-2014-50983"]]
```

Out[45]:

| Row ID | Order ID | Segment | Category | Category (OLD) | Sub-Category | Product Name | Product ID | Co |
|---|---|---|---|---|---|---|---|---|
| **AG-2014-50983** | AG-2014-2760 | Consumer | FURNITURE | NaN | CHAIRS | Novimex Rocking Chair, Black | FUR-NOV-10002453 | A |

1 rows × 34 columns

In [46…
```
df.loc["AG-2014-50983"]
```

Out[46]:

| | AG-2014-50983 |
|---|---|
| **Order ID** | AG-2014-2760 |
| **Segment** | Consumer |
| **Category** | FURNITURE |
| **Category (OLD)** | NaN |
| **Sub-Category** | CHAIRS |
| **Product Name** | Novimex Rocking Chair, Black |
| **Product ID** | FUR-NOV-10002453 |
| **Country** | Algeria |
| **Market** | Africa |
| **Region** | Africa |
| **Quantity** | 4.0 |
| **Discount** | 0.0 |
| **Profit** | 61.92 |
| **Customer ID** | CL-2565 |
| **Customer Name** | Clay Ludtke |
| **Order Priority** | High |
| **Postal Code** | 41244.0 |
| **Ship Mode** | Standard Class |
| **Shipping Cost** | 46.75 |
| **10/1/2014** | NaN |
| **7/1/2014** | NaN |
| **11/1/2014** | NaN |
| **9/1/2014** | NaN |
| **1/1/2014** | NaN |
| **12/1/2014** | NaN |
| **8/1/2014** | NaN |
| **5/1/2014** | NaN |
| **3/1/2014** | NaN |
| **4/1/2014** | NaN |
| **2/1/2014** | NaN |
| **6/1/2014** | 516.0 |
| **City** | Saida |

| | AG-2014-50983 |
|---|---|
| **State** | Saida |
| **manufacturers** | [Dunder Mifflin, Umbrella Corporation] |

**dtype:** object

In [47…

```
df.loc["AG-2014-50983", :]
```

Out[47]:

| | AG-2014-50983 |
|---|---|
| **Order ID** | AG-2014-2760 |
| **Segment** | Consumer |
| **Category** | FURNITURE |
| **Category (OLD)** | NaN |
| **Sub-Category** | CHAIRS |
| **Product Name** | Novimex Rocking Chair, Black |
| **Product ID** | FUR-NOV-10002453 |
| **Country** | Algeria |
| **Market** | Africa |
| **Region** | Africa |
| **Quantity** | 4.0 |
| **Discount** | 0.0 |
| **Profit** | 61.92 |
| **Customer ID** | CL-2565 |
| **Customer Name** | Clay Ludtke |
| **Order Priority** | High |
| **Postal Code** | 41244.0 |
| **Ship Mode** | Standard Class |
| **Shipping Cost** | 46.75 |
| **10/1/2014** | NaN |
| **7/1/2014** | NaN |
| **11/1/2014** | NaN |
| **9/1/2014** | NaN |
| **1/1/2014** | NaN |
| **12/1/2014** | NaN |
| **8/1/2014** | NaN |
| **5/1/2014** | NaN |
| **3/1/2014** | NaN |
| **4/1/2014** | NaN |
| **2/1/2014** | NaN |
| **6/1/2014** | 516.0 |
| **City** | Saida |

| | AG-2014-50983 |
|---|---|
| **State** | Saida |
| **manufacturers** | [Dunder Mifflin, Umbrella Corporation] |

**dtype:** object

In [48…
```
# Mean profit when Category is Furniture
df.loc[df["Category"] == "FURNITURE", "Profit"].mean()
```

Out[48]:
```
np.float64(26.68421359426352)
```

In [49…
```
df.loc[df["Category"] == "FURNITURE", "Profit"]
```

Out[49]:

| Row ID | Profit |
|---|---|
| **IN-2014-23218** | 39.840 |
| **IN-2014-24599** | 102.420 |
| **IN-2014-24597** | 79.680 |
| **IN-2014-27993** | 848.700 |
| **IN-2014-28967** | 356.580 |
| **…** | … |
| **ZA-2014-49186** | 16.800 |
| **ZI-2014-43446** | -181.458 |
| **ZI-2014-42718** | -109.038 |
| **ZI-2014-43714** | -1087.212 |
| **ZI-2014-44558** | -56.277 |

3347 rows × 1 columns

**dtype:** float64

In [50…
```
# profit is negative and market is EU
len(df.loc[(df["Profit"] < 0) & (df["Market"] == "EU")])
```

Out[50]:
```
768
```

# Note

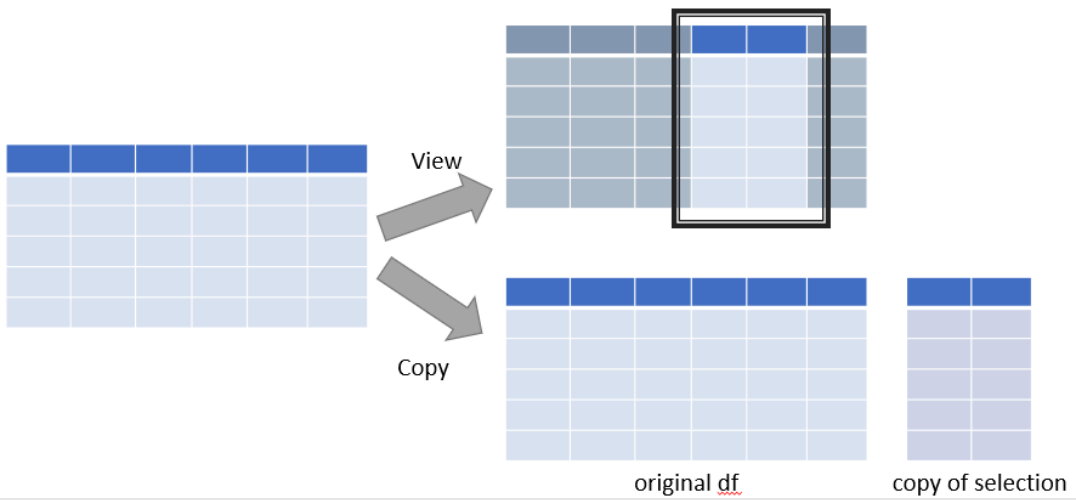
## Views vs copies

We often want to work with only a subset of a DataFrame. For that purpose, we can select only those rows or columns that we need and leave the rest.

Building off of our knowledge of views vs copies that we learned with our numpy tutorial, when we subset an array the result is not always a new array; sometimes what numpy returns is a view of the data in the original array. Since pandas Series and DataFrames are backed by numpy arrays, it will probably come as no surprise that something similar sometimes happens in pandas. Unfortunately, while this behavior is relatively straightforward in numpy, in pandas there's just no getting around the fact that it's a hot mess.

**The View/Copy Headache in pandas**: In numpy, the rules for when you get views and when you don't are a little complicated, but they are consistent: certain behaviors (like simple indexing) will always return a view, and others (fancy indexing) will never return a view.

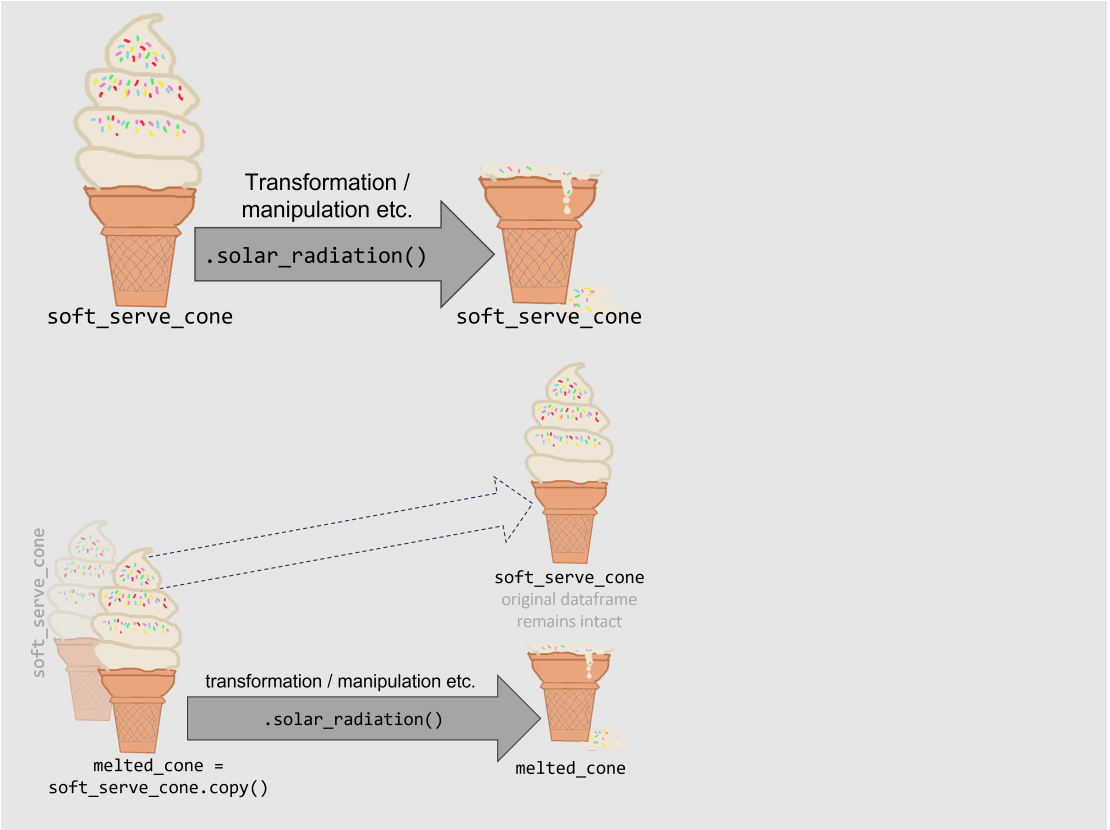
But in pandas, whether you get a view or not- and whether changes made to a view will propagate back to the original DataFrame- depends on the structure and data types in the original DataFrame




**When to create a copy using `.copy()` Creating copies** of DataFrame can be especially useful when doing exploratory analysis on a DataFrame where you want to keep the integrity of the original DataFrame in case you make a mistake.

`new_df = df.copy()` makes a copy (by creating a new object) of this object's indices and data. By default modifications to the data or indices of the copy will not be reflected in the original object but see the documentation and the parameter `deep` for more information

Note: Like all other variables, try to keep your DataFrame naming descriptive and intuitive to read! (For example "new_df" would be a bad name)




---

# Data Manipulations

- adjust column type
- create new columns
  - `.apply()`
- drop unnecessary or redundant columns
- sort the whole dataframe based on one column's values

### Change column data type

After examining your DataFrame you may find that there are certain columns whose data types do not match your assumptions or whose data types could inhibit analysis. For example, if you have a column that contains numbers, but the data type is string - you would not actually be able to do numeric operations on it. You can cast the data to a different datatype using **.astype()**.

To cast a column to a new data type: `df['column_1'] = df['column_1'].astype(new type)`

Remember some of the possible datatypes are:

- int
- float64
- bool
- string

In [51…
```
# we know that in our dataframe quantity should be an integer
# but right now it's a float

df["Quantity"]
```

Out[51]:

| Row ID | Quantity |
|---|---|
| IN-2014-23218 | 2.0 |
| IN-2014-24599 | 2.0 |
| IN-2014-24597 | 4.0 |
| IN-2014-27993 | 5.0 |
| IN-2014-28967 | 7.0 |
| … | … |
| ZA-2014-49187 | 1.0 |
| ZI-2014-42069 | 1.0 |
| ZI-2014-43712 | 1.0 |
| ZI-2014-48372 | 2.0 |
| ZI-2014-48014 | 1.0 |

17531 rows × 1 columns

**dtype:** float64

In [52…
```
# let's change that using .astype()
# then we can check the type after

# change the type to an int
df["Quantity"] = df["Quantity"] .astype(int)

# check the data type
df["Quantity"]
```

Out[52]:

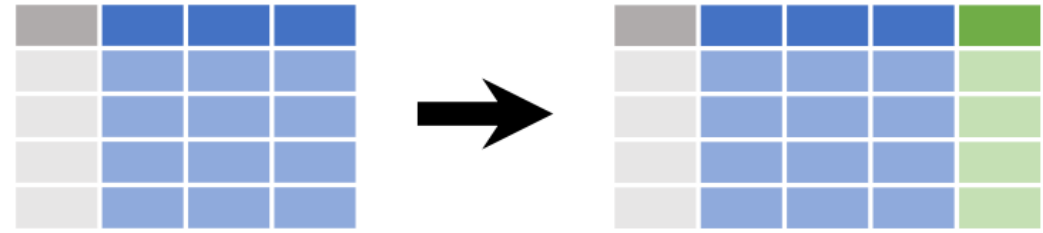
| Row ID | Quantity |
|---|---|
| IN-2014-23218 | 2 |
| IN-2014-24599 | 2 |
| IN-2014-24597 | 4 |
| IN-2014-27993 | 5 |
| IN-2014-28967 | 7 |
| ... | ... |
| ZA-2014-49187 | 1 |
| ZI-2014-42069 | 1 |
| ZI-2014-43712 | 1 |
| ZI-2014-48372 | 2 |
| ZI-2014-48014 | 1 |

17531 rows × 1 columns

**dtype:** int64

## Add new columns

Frequently you'll want to add **new columns**. There are multiple ways to do this!



To add already computed data:

- `df['new_column_name'] = already_computed_data`

You can also calculate new columns based on currently existing columns:

- `df['AB_SUM'] = df['A'] + df['B']`
- `df['Volume'] = df['Length'] * df['Height'] * df['Depth']`

Note: newly created columns are always added to the end of the DataFrame

Solution

In [53…
```
# let's create a "Profit Per Unit" column by diving profit by quantity

df['Profit Per Unit'] = df['Profit'] / df['Quantity']
```

In [54…
```
# now let's view the results

df[["Profit", "Quantity", "Profit Per Unit"]]
```

Out[54]:

| Row ID | Profit | Quantity | Profit Per Unit |
|---|---|---|---|
| **IN-2014-23218** | 39.840 | 2 | 19.920 |
| **IN-2014-24599** | 102.420 | 2 | 51.210 |
| **IN-2014-24597** | 79.680 | 4 | 19.920 |
| **IN-2014-27993** | 848.700 | 5 | 169.740 |
| **IN-2014-28967** | 356.580 | 7 | 50.940 |
| **…** | … | … | … |
| **ZA-2014-49187** | 19.710 | 1 | 19.710 |
| **ZI-2014-42069** | -20.799 | 1 | -20.799 |
| **ZI-2014-43712** | -59.562 | 1 | -59.562 |
| **ZI-2014-48372** | -93.180 | 2 | -46.590 |
| **ZI-2014-48014** | -20.298 | 1 | -20.298 |

17531 rows × 3 columns

### .apply()

**.apply()** is a fairly powerful tool that can do many things. In this context we will use it to create new columns using functions from other packages or functions you've created.

- `df['new_column_name'] = df[column(s) of interest].apply(function)`
- `df['A_sqrt'] = df['A'].apply(np.sqrt)`
- `df['A_adjusted'] = df['A'].apply(your_function)`

In [55…
```
# let's create a function to fix the capitalization for strings in this c

df['Category']
```

Out[55]:

| Row ID | Category |
|---|---|
| **IN-2014-23218** | FURNITURE |
| **IN-2014-24599** | FURNITURE |
| **IN-2014-24597** | FURNITURE |
| **IN-2014-27993** | FURNITURE |
| **IN-2014-28967** | FURNITURE |
| **…** | … |
| **ZA-2014-49187** | TECHNOLOGY |
| **ZI-2014-42069** | TECHNOLOGY |
| **ZI-2014-43712** | TECHNOLOGY |
| **ZI-2014-48372** | TECHNOLOGY |
| **ZI-2014-48014** | TECHNOLOGY |

17531 rows × 1 columns

**dtype:** object

In [56…
```
# let's practice by making a function that will fix the capitalization of
# "Category" column and then applying it

def capitalize_first_letter(string):
    all_lower = string.lower()
    cap_first = all_lower.capitalize()
    return cap_first

df["Category"] = df["Category"].apply(capitalize_first_letter)

# view the results
df[["Category"]]
```

Out[56]:

| Row ID | Category |
|---|---|
| **IN-2014-23218** | Furniture |
| **IN-2014-24599** | Furniture |
| **IN-2014-24597** | Furniture |
| **IN-2014-27993** | Furniture |
| **IN-2014-28967** | Furniture |
| **...** | ... |
| **ZA-2014-49187** | Technology |
| **ZI-2014-42069** | Technology |
| **ZI-2014-43712** | Technology |
| **ZI-2014-48372** | Technology |
| **ZI-2014-48014** | Technology |

17531 rows × 1 columns

**Exercises:**

1. In these code examples, what do you think the `axis` argument is doing? What kind of output do you think will be returned? For each?

- `df[['A', 'B']].apply(np.mean, axis = 0)`
- `df[['A', 'B']].apply(np.mean, axis = 1)`

2. Which axis setting is the default? (hint- check the documentation)

In [57…

```
### Your code here
```

Solutions

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.apply.html

axis{0 or 'index', 1 or 'columns'}, default 0 Axis along which the function is applied:

0 or 'index': apply function to each column.

1 or 'columns': apply function to each row.

In [58…

```
# test it out with a simple df

num_df = pd.DataFrame({
```

```
    "A": [10, 20, 30, 40],
    "B": [1, 2, 3, 4]
})
num_df
```

Out[58]:

| | A | B |
|---|---|---|
| **0** | 10 | 1 |
| **1** | 20 | 2 |
| **2** | 30 | 3 |
| **3** | 40 | 4 |

In [59…

```
# 1 value per column

num_df[['A', 'B']].apply(np.mean, axis = 0)
```

Out[59]:

| | 0 |
|---|---|
| **A** | 25.0 |
| **B** | 2.5 |

**dtype:** float64

In [60…

```
# 1 value per row

num_df[['A', 'B']].apply(np.mean, axis = 1)
```

Out[60]:

| | 0 |
|---|---|
| **0** | 5.5 |
| **1** | 11.0 |
| **2** | 16.5 |
| **3** | 22.0 |

**dtype:** float64

### Drop specific columns & rows using `.drop()`

You can easily drop rows or columns using **`.drop()`**

There are many reasons you may wish to drop columns from your DataFrame. Here are a few examples:

- irrelevant to the analysis you are going to do
- out of date
- redundant information
- non-analyzable due to high rates of NaN or empty entries

Drop columns:

- `df = df.drop(columns=['column1', 'columns2'])`
- `df = df.drop('column_name',axis=1)`

Drop rows:

- `df = df.drop(index=('index1_string, 'index2_string'))`
- `df = df.drop([row, row])`

In [61…
```
# let's drop the 'Category (OLD)'column

df.drop(columns=["Category (OLD)"], inplace = True)

# check the results in the dataframe columns list
df.columns
```

Out[61]:
```
Index(['Order ID', 'Segment', 'Category', 'Sub-Category', 'Product Nam
e',
       'Product ID', 'Country', 'Market', 'Region', 'Quantity', 'Discoun
t',
       'Profit', 'Customer ID', 'Customer Name', 'Order Priority',
       'Postal Code', 'Ship Mode', 'Shipping Cost', '10/1/2014', '7/1/20
14',
       '11/1/2014', '9/1/2014', '1/1/2014', '12/1/2014', '8/1/2014',
       '5/1/2014', '3/1/2014', '4/1/2014', '2/1/2014', '6/1/2014', 'Cit
y',
       'State', 'manufacturers', 'Profit Per Unit'],
      dtype='object')
```

## Sort the DataFrame based on column values

`.sort_values()` to sort your DataFrame by the the values of a particular column or columns.

`df.sort_values(by=['column1', 'column2'], ascending = boolean, inplace = boolean)`

Pro-tip: Sorting the values of your DataFrame before plotting creates more interpretable plots.

In [62…

```
# let's sort our USA_df by 'Segment', 'Category' and 'Sub-Category'
# we will leave the underlying data unchanged by setting inplace to False

df.sort_values(by = ["Segment", "Category", "Sub-Category"], inplace = Fa
```

Out[62]:

| Row ID | Order ID | Segment | Category | Sub-Category | Product Name | Prod |
|---|---|---|---|---|---|---|
| **AG-2014-46003** | AG-2014-8600 | Consumer | Furniture | BOOKCASES | Sauder Classic Bookcase, Pine | FU SA 10002 |
| **AO-2014-45199** | AO-2014-9370 | Consumer | Furniture | BOOKCASES | Bush Classic Bookcase, Pine | FU BU 10001 |
| **US-2014-6344** | US-2014-165512 | Consumer | Furniture | BOOKCASES | Bush Library with Doors, Metal | FUR-B 10002 |
| **US-2014-1158** | US-2014-107664 | Consumer | Furniture | BOOKCASES | Ikea Floating Shelf Set, Traditional | FUR-B 10001 |
| **IN-2014-21802** | IN-2014-25644 | Consumer | Furniture | BOOKCASES | Bush Corner Shelving, Metal | FUR-B 10004 |
| **...** | ... | ... | ... | ... | ... | |
| **US-2014-40156** | US-2014-114356 | Home Office | Technology | PHONES | Plantronics Encore H101 Dual Earpieces Headset | TEC-P 10003 |
| **US-2014-33227** | US-2014-101539 | Home Office | Technology | PHONES | Mitel MiVoice 5330e IP Phone | TEC-P 10004 |
| **CA-2014-36882** | CA-2014-105333 | Home Office | Technology | PHONES | Panasonic Business Telephones KX-T7736 | TEC-P 10001 |
| **ID-2014-26472** | ID-2014-41597 | Home Office | Technology | PHONES | Nokia Speaker Phone, Cordless | TEC-P 10001 |
| **ID-2014-26474** | ID-2014-41597 | Home Office | Technology | PHONES | Samsung Signal Booster, with Caller ID | TEC-P 10003 |

17531 rows × 34 columns

**Exercise:**

1. Sort the df by profit per unit

In [63…
```
### Your code here
```

Solutions

In [64…
```
df["Profit per Unit"] = df["Profit"] / df["Quantity"]
df_sorted = df.sort_values("Profit per Unit", ascending = False)
df_sorted
```

Out[64]:

| Row ID | Order ID | Segment | Category | Sub-Category | Product Name | Product ID | Country |
|---|---|---|---|---|---|---|---|
| **CA-2014-35487** | CA-2014-166709 | Consumer | Technology | COPIERS | Canon imageCLASS 2200 Advanced Copier | TEC-CO-10004722 | United States |
| **CA-2014-39450** | CA-2014-140151 | Consumer | Technology | COPIERS | Canon imageCLASS 2200 Advanced Copier | TEC-CO-10004722 | United States |
| **CA-2014-33920** | CA-2014-127180 | Home Office | Technology | COPIERS | Canon imageCLASS 2200 Advanced Copier | TEC-CO-10004722 | United States |
| **CA-2014-37817** | CA-2014-138289 | Consumer | Office supplies | BINDERS | GBC DocuBind P400 Electric Binding System | OFF-BI-10004995 | United States |
| **SF-2014-48905** | SF-2014-4490 | Consumer | Furniture | TABLES | Bevis Conference Table, Rectangular | FUR-BEV-10004805 | South Africa |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **CA-2014-36607** | CA-2014-131254 | Consumer | Office supplies | BINDERS | Fellowes PB500 Electric Punch Plastic Comb Bin... | OFF-BI-10003527 | United States |
| **CA-2014-34143** | CA-2014-152093 | Home Office | Office supplies | BINDERS | Fellowes PB500 Electric Punch Plastic Comb Bin... | OFF-BI-10003527 | United States |
| **US-2014-36288** | US-2014-122714 | Corporate | Office supplies | BINDERS | Ibico EPK-21 Electric Binding System | OFF-BI-10001120 | United States |
| **CA-2014-34308** | CA-2014-134845 | Home Office | Technology | MACHINES | Lexmark MX611dhe Monochrome Laser Printer | TEC-MA-10000822 | United States |

| Row ID | Order ID | Segment | Category | Sub-Category | Product Name | Product ID | Country |
|---|---|---|---|---|---|---|---|
| **US-2014-31980** | US-2014-168116 | Corporate | Technology | MACHINES | Cubify CubeX 3D Printer Triple Head Print | TEC-MA-10004125 | United States |

17531 rows × 35 columns

**Final Exercises: Putting It All Together**

1. How many rows and columns are in df, and what are the unique values in the Region column?
2. Select all rows where Region is West and Order Priority is Critical. How many orders match?
3. Create a new column Shipping Cost Per Unit (Shipping Cost ÷ Quantity). What's the average value across the dataset?
4. Using `.iloc`, select the first 10 rows and columns 3 through 5 (by position) of df. Then check `.columns` to confirm which columns those actually are.
5. Sort df by Profit (descending), drop the Postal Code column, and show the top 5 highest-profit orders with only Category, Sub-Category, and Profit.

Solutions

In [65…
```
# Q1

print(df.shape)
df['Region'].unique()
```

```
(17531, 35)
```

Out[65]:
```
array(['Central Asia', 'Africa', 'South', 'Oceania', 'Central', 'EMEA',
       'Caribbean', 'Southeast Asia', 'Canada', 'North Asia', 'North',
       'West', 'East'], dtype=object)
```

In [66…
```
# Q2

critical_west = df.loc[(df['Region'] == 'West') & (df['Order Priority'] =
len(critical_west)
```

Out[66]: 68

In [67…
```
# Q3

df['Shipping Cost Per Unit'] = df['Shipping Cost'] / df['Quantity']
df['Shipping Cost Per Unit'].mean()
```

Out[67]: np.float64(7.8151359787604635)

In [68…
```
# Q4

df.iloc[:10, 2:5]
df.columns[2:5]
```

Out[68]: Index(['Category', 'Sub-Category', 'Product Name'], dtype='object')

In [69…
```
# Q5

df_sorted = df.sort_values('Profit', ascending=False)
df_sorted.drop(columns=['Postal Code'])[['Category', 'Sub-Category', 'Pro
```

Out[69]:

| Row ID | Category | Sub-Category | Profit |
|---|---|---|---|
| **CA-2014-39450** | Technology | COPIERS | 6719.9808 |
| **CA-2014-35487** | Technology | COPIERS | 5039.9856 |
| **ES-2014-12069** | Office supplies | APPLIANCES | 3979.0800 |
| **CA-2014-33920** | Technology | COPIERS | 3919.9888 |
| **MO-2014-50788** | Technology | COPIERS | 2597.2800 |